CanDIAPL Python development guide

Guide written by Erik Osinga & Connor Stone, with inspiration from the [LSST Data Management Style Guides](), the [Scientific Python standard](), and ChatGPT.

Introduction & Scope

These docs assume you're running Linux; if you're running macOS, you're probably aware of the things you have to change when following these instructions. If you're running Windows, you should probably fix that.

This document is accompanied by a template repository located in the CANDIAPL GitHub organisation. You can easily use the template as a starting point when designing your own software. Check it out at: https://github.com/CanDIAPL/candiamazing/

If you've already read these docs thoroughly, a summary of the commands to set up a scientific Python project can be found in the quickstart here.

This is supposed to be a very minimal set of instructions for writing scientific software for CANDIAPL, aimed at astronomers who are proficient in Python, but not necessarily up to date on coding standards. An extensive coverage of best practices can be found in the Scientific Python standard.

Motivation: why go to all the trouble?

A quick scroll through this document (and the Candiamazing template repo) will show you that it is a lot of work to set up a project the way we are describing. Why go to all this trouble when you could just open a folder and start writing code to get results? While just writing some code now would probably get you *something* faster, the layout we propose will save you a huge number of headaches if your goal is reliable and widely usable code. Here is a small list of the issues we've found to be solved by following this project layout:

- Packaging for pip/conda. This layout is what these package distributors expect, so publishing your code to the community will be seamless. This prevents you from writing a package that only runs on your own machine, and will allow you to run it years after your code was written.
- Rooting out minor typos, ever mixed up "i" and "j" in your code? *Ruff* and *MyPy* are on the lookout for these problems
- Ever needed to make a minor change in a core function and realised that suddenly everything stopped working? Unit tests catch these issues before they mess up your data.
- Ever had code work on your laptop but not on a cluster? GitHub Actions unit tests build a fresh environment before running, so a missing package gets found early, before you move to production or cluster-scale computing.
- If lots of people are using your code (which is the goal!), you can get inundated with questions unless you have clear and detailed documentation. This is a problem best solved before it happens, since your free time can really get sucked up by answering a flood of questions! Additionally, "future you" will be incredibly thankful for clear documentation.
- As a project grows in scale, it becomes increasingly important that you are able to focus on a small chunk of the code without needing to keep in your head how it interacts with everything else. Modularisation is key for this, and unit tests are a must to ensure nothing gets broken down the line
- Without agreed-on expectations and standards, working with other people on the same codebase becomes tricky, and code review becomes very subjective. A small, explicit set of enforced rules keeps collaboration working smoothly!

Project layout and packaging basics

Quick summary

- Always use a virtual environment for developing code, like `conda` or `venv`.
- Use `pyproject.toml` to package your code, following the recommended directory structure as explained in this section.

Virtual environment

We recommend always developing and testing code in a virtual environment.

The easiest way to set up a virtual environment is to use `venv` , which comes with Python:

```
~$ python3 -m venv .my_new_venv
```

Which will create a virtual environment in the current folder, called `.my_new_venv`
You only need to do this once. Then activate the virtual environment with

```
~$ source .my_new_venv/bin/activate
```

Another option is to create a conda environment, which is useful if your code depends on software that is not easily installable via `pip`. You can use miniconda, for example.

Project structure

General project structure guidelines

- A `src/<project_name>` directory with the source code, split into decently named sub-modules (config.py, core.py, cli.py, etc)
- Have a `tests/` directory with test scripts denoted as `test_*py`

Example project structure

Let's say we're creating a project called `candiamazing`

```
~$ mkdir candiamazing
~$ cd candiamazing
```

First, initialise a Git repository

```
~$ git init
```

Add a standard `.gitignore` file for Python

```
~$ wget https://raw.githubusercontent.com/github/gitignore/refs/heads/main/Python.gitignore -o .gitignore
```

Optionally, you can connect this project to an existing GitHub repository using

```
~$ git remote add origin git@github.com:<username>/repository.git
```

Or set up a GitHub repository from scratch (see Section initialise GitHub from CLI)

Then, let's create a virtual environment for this software

```
~$ python -m venv .camazing_venv
~$ source .camazing_venv/bin/activate
```

A typical project structure looks as follows:

```
.
└── candiamazing
    ├── pyproject.toml
    ├── README.md
    └── src
        └── candiamazing
            └── __init__.py
```

Create this, e.g. with

```
~$ cd candiamazing # should be already done above
~$ touch pyproject.toml
~$ touch README.md
~$ mkdir -p src/candiamazing/
~$ touch src/candiamazing/__init__.py
```

Alternatively to creating the project structure manually, you can also use the Python package and project manager uv, an extremely fast and easy-to-use project manager. Install the uv software (e.g. pip install uv), and initialise the project with

```
pip install uv
uv init --package candiamazing
```

This should have created the same directory structure as above.

The project configuration is defined in `pyproject.toml`, which minimally should look like this

```
[project]
name = "candiamazing"
```

```
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
authors = [
    { name = "Connor Stone", email = "connor.stone@utoronto.ca" }
    { name = "ErikOsinga", email = "erikosinga876@hotmail.com" }
]
requires-python = ">=3.11"
dependencies = [“numpy>=2.3”]

[project.scripts]
candiamazing = "candiamazing:main"

[build-system]
requires = ["hatchling", "hatch-vcs"]
build-backend = "hatchling.build"
```

If needed, you can change the default settings in this file. See https://packaging.python.org/en/latest/guides/writing-pyproject-toml/

To add a dependency to our Python package, we add it to the “dependencies” list in the pyproject.toml file:

```
[project]
name = "candiamazing"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
authors = [
    { name = "ErikOsinga", email = "erikosinga876@hotmail.com" }
]
requires-python = ">=3.11"
dependencies = [
    "numpy>=2.3",
    ”scipy”,
]

[project.scripts]
candiamazing = "candiamazing:main"

[build-system]
requires = ["hatchling", "hatch-vcs"]
build-backend = "hatchling.build"
```

At this point, you should be able to install the `candiamazing` package in “editable” mode

```
~$ cd candiamazing
~$ pip install -e .
```

Confirm that this worked with

```
~$ pip show candiamazing

Name: candiamazing
Version: 0.1.0
Summary: Add your description here
Home-page:
Author: ErikOsinga
Author-email: ErikOsinga <erikosinga876@hotmail.com>
License:
Location:
/home/osingae/OneDrive/postdoc/POSSUM/candiapl-example-software/.venv/lib/python3.1
1/site-packages
Editable project location:
/home/osingae/OneDrive/postdoc/POSSUM/candiapl-example-software/candiamazing
Requires: numpy
Required-by:
```

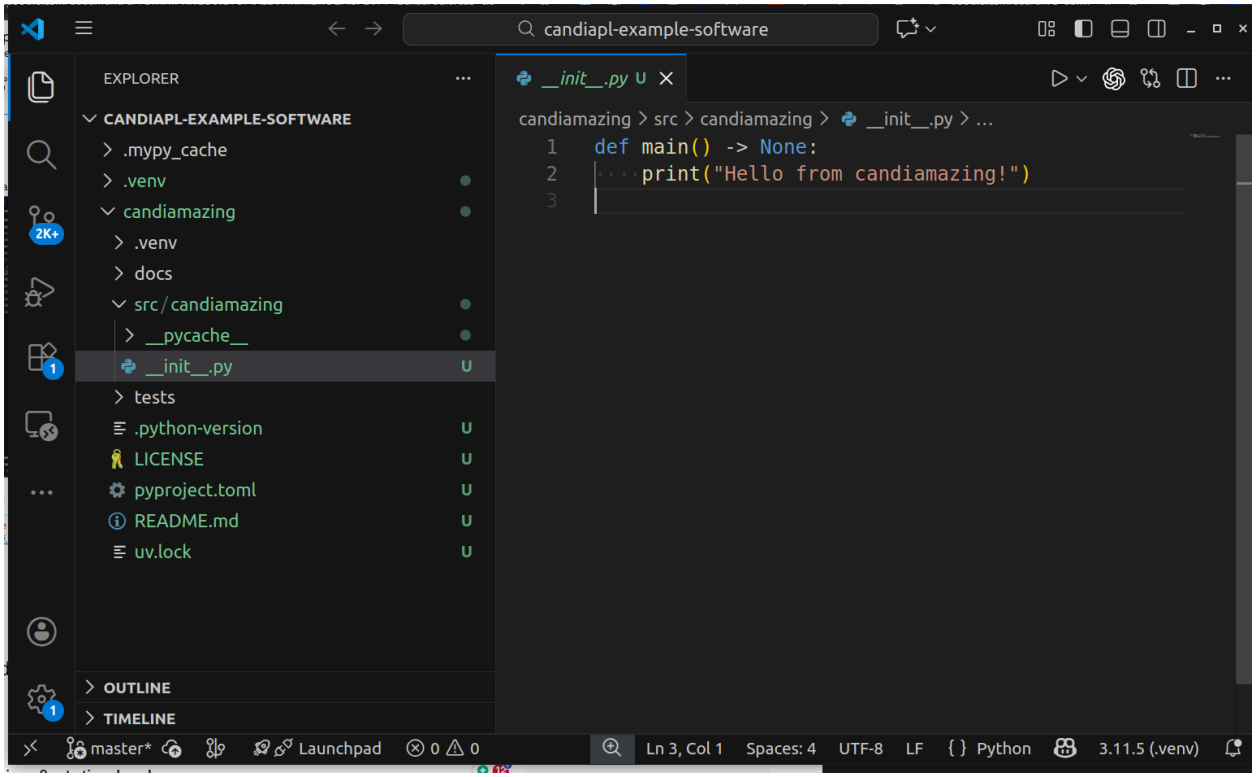
You have now created a barebones CANDIAPL package that you can import, see e.g.

```
~$ python -c "import candiamazing"
```

Let’s add a directory for tests and docs, and a LICENSE file for later.

```
~$ mkdir tests
~$ mkdir docs
~$ touch LICENSE
```

An example VSCode window while working on this project would look like



Module naming

Maybe we could note some recommended but not enforced naming conventions for modules e.g. *cli.py for command line interfaces, core.py for algorithms, config.py for configurations, etc*

Module naming example

Suppose that we are in need of a flux to magnitude converter, and would like this `candiamazing` package to calculate these conversions for us.

Core functionality should go into its own file; this is where we define a basic interface for the user. Create a `core.py` module

```
touch src/candiamazing/core.py
```

We'll have the user input parameters like zeropoint to help complete the conversion.
We can place utilities, such as the math behind actually converting fluxes, in a utils file. This handles functional, small scope code that is sometimes very reusable.

```
touch src/candiamazing/utils.py
```

And finally, create `cli.py` for a command-line interface.

```
touch src/candiamazing/cli.py
```

Assuming that we have filled these with the required code (see [GitHub example](), we can now execute our code in the familiar way with

```
python src/candiamazing/cli.py
```

However, the project setup also makes it easy to create an executable script by editing the `[project.scripts]` section in the `pyproject.toml` to the following:

```
[project]
name = "candiamazing"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
authors = [
   { name = "ErikOsinga", email = "erikosinga876@hotmail.com" }
]
requires-python = ">=3.11"
dependencies = [
   "astropy>=7.2.0",
   "numpy>=2.3.5",
]

[project.scripts]
candiamazing = "candiamazing.cli:main"


[build-system]
requires = ["hatchling", "hatch-vcs"]
build-backend = "hatchling.build"
```

Now, upon installing the `candiamazing` software, we will have the executable program `candiamazing` which executes the function `main()` in `candiamazing/cli.py`

Re-install the changed package:
`uv pip install -e .`

And the CLI `candiamazing` is now available as an executable:

```
connor@carl:~/Programming/candiamazing$ source ~/venvs/camazing/bin/activate
(camazing) connor@carl:~/Programming/candiamazing$ candiamazing mag_to_flux 10 8.9
0.36307805477010147
(camazing) connor@carl:~/Programming/candiamazing$ candiamazing --help
usage: candiamazing [-h] {flux_to_mag,mag_to_flux} ...

Convert between fluxes and magnitudes using a simple magnitude system.

positional arguments:
  {flux_to_mag,mag_to_flux}
                        Available commands
    flux_to_mag         Convert flux to magnitude
    mag_to_flux         Convert magnitude to flux

options:
  -h, --help            show this help message and exit

Examples:
  candiamazing flux_to_mag 3631 8.9
  candiamazing mag_to_flux 10 8.9

Run `candiamazing <command> -h` for command-specific help.
(camazing) connor@carl:~/Programming/candiamazing$
```

The project file tree now looks like this, from the `candiamazing` directory.

```
.
├── config_example.toml
├── docs
├── LICENSE
├── pyproject.toml
├── README.md
├── src
│   └── candiamazing
│       ├── cli.py
│       ├── config.py
│       ├── core.py
│       ├── __init__.py
│       └── __pycache__
│           ├── cli.cpython-311.pyc
│           ├── config.cpython-311.pyc
│           ├── core.cpython-311.pyc
│           └── __init__.cpython-311.pyc
├── tests
└── uv.lock
```

Code style & formatting

Quick summary

- We follow the PEP 8 style guide.
- Use an auto-formatter/linter, recommending *Ruff*
- Some basic naming conventions: functions/variables: `lower_case_underscores`, and classes: `CamelCaseAllWords`
- Use type hinting as much as possible (see here)

We'll follow PEP 8 as a baseline style guide, though not all of PEP 8 will be strictly enforced. You don't need to read the whole guide, though, as most of it can be enforced automatically by a *Linter*. A linter is a static code analysis program that flags bugs or style errors before you even run the code. Any CanDIAPL project should use Ruff as the default linter, as it is easily configurable via the `pyproject.toml` file.

Still, some conventions are worth remembering, like consistent naming of objects:

- modules and packages named as `lowercase_with_underscores` (e.g. `candiamazing`, `core.py`
- functions and methods named as `lowercase_with_underscores` (e.g. `flux_to_mag`);
- classes: `CapitalWords` (e.g. `BrightnessConverter`);
- constants: `UPPERCASE_WITH_UNDERSCORES` (e.g. `RM_COEFFICIENT = 0.812`).

Linting + Formatting

Ruff is the only linter+formatter we standardise on. It is fast, works well with `pyproject.toml` and covers a wide range of rule sets. You can add **Ruff** as a development-only dependency by adding the following lines to `pyproject.toml`

```
[project.optional-dependencies]
dev = ["ruff"]
```

This adds a dependency that can be optionally installed if you want to develop for a package, with the [dev] flag

```
pip install -e .[dev]
```

Then add this minimal Ruff configuration to `pyproject.toml`

```
[tool.ruff]
# Apply to both src and tests by default
src = ["src", "tests"]
# default line length
```

```
line-length = 100

[tool.ruff.format]
# use double quotes for strings.
quote-style = "double"
# indent with spaces, rather than tabs.
indent-style = "space"
# respect magic trailing commas.
skip-magic-trailing-comma = false

[tool.ruff.lint]
# Can expand this as far as user wants
select = [
 "E",   # pycodestyle (PEP8) errors
 "F",   # pyflakes
 "I",   # isort-like import sorting
 "B",   # bugbear (common bugs)
 "UP",  # pyupgrade (modern Python idioms)
]
```

You should now be able to run `ruff check` from the directory with the pyproject.toml file to run the linter and `ruff format` to run the formatter.

```
(.venv) osingae:~/OneDrive/postdoc/POSSUM/candiapl-example-software/candiamazing$ ruff check
I001 [*] Import block is un-sorted or un-formatted
 --> src/candiamazing/config.py:1:1
  |
1 | / from dataclasses import dataclass
2 | | from pathlib import Path
3 | | import tomllib
4 | |
5 | | from astropy import units as u
  | |_______________________________^
  |
help: Organize imports

Found 1 error.
[*] 1 fixable with the `--fix` option.
```

This shows that our import block is not sorted as Ruff would like. We can simply fix this by running `ruff check --fix`

```
(.venv) osingae:~/OneDrive/postdoc/POSSUM/candiapl-example-software/candiamazing$ ruff check --fix
Found 1 error (1 fixed, 0 remaining).
```

The formatter checks the code format, in this case catching that one line is longer than 88 characters, and adding some good-practice whitespace. See `ruff format --diff`

```
(.venv) osingae:~/OneDrive/postdoc/POSSUM/candiapl-example-software/candiamazing$ ruff format --diff
--- src/candiamazing/cli.py
+++ src/candiamazing/cli.py
@@ -30,7 +30,9 @@
     )

     # Astropy quantities support formatting with f-strings
-    print(f"Electron density: {cfg.electron_density.value} [{cfg.electron_density.unit}]")
+    print(
+        f"Electron density: {cfg.electron_density.value} [{cfg.electron_density.unit}]"
+    )
     print(f"Magnetic field:   {cfg.magnetic_field.value} [{cfg.magnetic_field.unit}]")
     print(f"Path length:      {cfg.path_length.value} [{cfg.path_length.unit}]")
     print("---")

--- src/candiamazing/config.py
+++ src/candiamazing/config.py
@@ -9,13 +9,14 @@
 class RMConfig:
     """
     Dataclass to hold rotation measure configuration parameters.
-
+
     Parameters
     ----------
         electron_density: Free electron density as an astropy Quantity.
         magnetic_field: Magnetic field component along the line of sight as an astropy Quantity.
         path_length: Path length along the line of sight as an astropy Quantity.
     """
+
     electron_density: u.Quantity
     magnetic_field: u.Quantity
     path_length: u.Quantity
@@ -46,4 +47,3 @@
         magnetic_field=b,
         path_length=length,
     )
-

2 files would be reformatted, 2 files already formatted
```

If we're happy with that, we can run `ruff format`

In summary, `ruff` is there to catch:

- General PEP-8 consistent formatting
- obvious bugs (unused variables, missing imports)
- questionable constructs (e.g. dead code)
- style and import-order problems

If Ruff complains and you think it is wrong, the default is to **fix the code**. If you are confident the rule is not appropriate, you can disable it locally with a comment, e.g. disabling the Ruff **I001 import error** for the first line can be done with

```
from dataclasses import dataclass # noqa: I001
```

But please do that sparingly!

Static type checking

We *strongly encourage*, but don't enforce, type hinting when writing code. This enhances readability and enables static type checkers, like MyPy (see https://learn.scientific-python.org/development/guides/mypy/), that are incredibly powerful for finding bugs during development.

Essentially, type hinting means that you declare the expected input and output types. For example:

```
def f(x: int) -> int:
    return x * 5
```

Let's add MyPy to the development dependencies

```
[project.optional-dependencies]
dev = ["ruff", "mypy"]
```

And re-install our package with it

```
pip install -e .[dev]
```

In CANDIAPL, we recommend a default configuration as follows:

```
[tool.mypy]
packages = ["candiamazing"]
ignore_missing_imports = true
strict = false
```

- `ignore_missing_imports = true` keeps things manageable when working with third-party packages that don't fully support typing, like Astropy. Otherwise, we would need to add a `#type: ignore` comment behind every astropy import:

  ```
  from astropy import units as u  # type: ignore
  ```

- We set `strict = false` to avoid MyPy complaining for every function that doesn't contain a type annotation. If you want to go hard on typing, feel free to set this to `true`.

Now, simply type check your files using `mypy .` in the `candiamazing` module directory

```
(.venv) osingae:~/OneDrive/postdoc/POSSUM/candiapl-example-software/candiamazing$ mypy .
Success: no issues found in 4 source files
```

Pre-commit

`pre-commit` is a framework for managing and maintaining a list of Git pre-commit hooks. These hooks are scripts that run automatically before every commit to check your code for issues like style violations, missing docstrings, or type errors. Using `pre-commit` helps enforce code standards locally, ensuring that only clean, well-formatted, and linted code makes it into your repository.

Setup Instructions

1. **Install `pre-commit`:**
   Add `pre-commit` to your dev dependencies in `pyproject.toml` and install it yourself with: `pip install pre-commit`

dev = ["ruff", "mypy", "pytest", "pytest-cov", "pre-commit"]

Then run:

```
pip install -e .[dev]
```

2. **Create a `.pre-commit-config.yaml` file:**
   This file defines which hooks should run. Create a file named `.pre-commit-config.yaml` in the root of your repository.

A minimal configuration to run `ruff` (for linting and formatting) and `mypy` (for static type checking) would look like this:

```
repos:

  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.1.6 # Use the latest Ruff release version
    hooks:
      - id: ruff
        args: [--fix, --exit-non-zero-on-fix]
      - id: ruff-format

  - repo: https://github.com/pre-commit/mirrors-mypy
```

```
    rev: v1.17.1 # Use a recent version
    hooks:
      - id: mypy
```

- The `ruff` hook runs linting and attempts to fix simple issues (`--fix`). `--exit-non-zero-on-fix` ensures that if Ruff makes any fixes, the commit is aborted, requiring you to stage and commit the changes.
- The `ruff-format` hook handles code formatting.
- The `mypy` hook runs the static type checker.

3. **Install the hooks into your Git repository:**
   Run the following command in your project's root directory:

```
pre-commit install
```

This command sets up the Git hook scripts. Now, every time you run `git commit`, `pre-commit` will execute the specified hooks (`ruff` and `mypy`) against the files you are trying to commit.

IDE

You're of course free to use whatever integrated development environment you prefer. Some popular ones for Python are *Spyder, PyCharm, and VSCode.*

After you've created & activated your virtual environment, you can probably point your favourite IDE to this environment. For example, VSCode can be pointed to it by pressing `Ctrl + Shift + P` and searching for "Python: Select Interpreter", and then selecting the `.venv/bin/python interpreter`

VSCode also has plugins to run **MyPy** and **Ruff** as you are developing, simply install these from the *Extensions* tab. Other IDEs might be able to do something similar. This will make your life much easier when developing code.

Documentation & docstrings

Quick summary:

- Documentation is important, for other people but also for future you.
- Use numpy style docstrings and keep the formatting constant throughout the package
- Higher-level documentation: A readthedocs (or GitHub Pages) website for complex projects, a detailed README for small packages/tools

Good documentation is part of doing reproducible science, not an optional extra.
In CANDIAPL projects, “documentation” lives at (at least) two levels:

1. Docstrings in the code (explaining the function/class/module)
2. README and examples (how to install and run the project).

A documentation string (docstring) is a string describing a module, function, class, or method definition. For consistency, docstrings are always surrounded by triple double quotes, i.e.:

```
"""This is the form of a docstring.

It can be spread over several lines.

"""
```

A NumPy-style docstring typically has:

- a **one-line summary**,
- an optional extended description,
- sections such as `Parameters`, `Returns`, `Raises`, `Notes`, `Examples`.

We rely on **type hints** for types, so docstrings should focus on:

- what the function *does*,
- the meaning and units of quantities,
- any assumptions or caveats,
- references and equations where relevant.

Docstring examples

For a simple function, this is an example of a good docstring

```python
from collections.abc import Iterable
def sum(values: Iterable) -> float:
    """Sum numbers in an array.

    Parameters
    ----------
    values : iterable
       Python iterable whose values are summed.

    Returns
    -------
    sum : `float`
       Sum of ``values``.
    """
    pass # write function here
```

Extremely simple functions can have a one-line summary only.

Modules and classes that form part of the public API should also have docstrings. Here's a Module-level docstring example at the top of `core.py`:

```python
"""A class for converting between flux and magnitude.

   This class converts brightness values between linear flux units and
   logarithmic magnitude units. The provided zeropoint defines the
   Reference level for the magnitude system. For example, in the AB
   magnitude system the zeropoint can be set to 8.9 when the fluxes are
   in units of Janskys where the reference flux would be 3631 Jy. In
   CGS units of flux density (erg s^-1 cm^-2 Hz^-1), the zeropoint would
   be set to 48.6.

   Parameters
   ----------
   zeropoint : float
       The zeropoint for the magnitude system.
"""
```

Existence and formatting of docstrings can also be checked automatically with `Ruff`. We add the "D" setting in pyproject.toml to enforce docstrings, and set the docstring type to "numpy":

```
[tool.ruff.lint]
select = [
  "E",   # pycodestyle (PEP8) errors
  "F",   # pyflakes
  "I",   # isort-like import sorting
  "B",   # bugbear (common bugs)
  "UP",  # pyupgrade (modern Python idioms)
  "D",   # pydocstyle: docstring rules
]
ignore = [
  "D107",  # missing docstring in __init__
]
[tool.ruff.lint.pydocstyle]
convention = "numpy"
```

If you've enabled `ruff` as e.g. a plugin in your IDE, it will run automatically during code development and tell you when you're missing a docstring. Otherwise, you can run `ruff` from the command line with

```
ruff check
```

For new CANDIAPL projects, turn on `ruff` with the above rules from day one. For already existing, messy codebases, you can add per-file ignores for legacy modules, marking known-bad areas as exempt:

```
# for existing modules we could set specific ignores
[tool.ruff.lint.per-file-ignores]
"src/legacy/**.py" = ["D"]
```

But please do so sparingly.

README and examples

Every CANDIAPL repository must have a `README.md` at the top level, containing at least:

- a short description of what the project does,
- installation instructions

a minimal example (code block) showing how to use the primary API or command-line interface.

Testing

Quick summary

- Requires tests to be covering at least 80% ?
- Use *Pytest*, a *tests/* directory with tests all named as *test*py* (should also have some example tests to demo pytest features)

Best practices for testing

Tests are there to make sure code is working as intended and to make refactoring safe. In CANDIAPL projects, we standardise on:

- `pytest` as the test runner,
- a dedicated **`tests/` directory** at the top level,
- test files named **`test_*.py`**.

The minimum expectation is:

- Functionality in `src/` comes with tests in `tests/`,
- `pytest` passes before merging,
- Tests are runnable with a single command.

In our example software, all tests live under `tests/` and are named referring to the file that they are testing.

```
└── candiamazing
    ├── config_example.toml
    ├── docs
    ├── LICENSE
    ├── pyproject.toml
    ├── README.md
    ├── src
    │   └── candiamazing
    │       ├── cli.py
    │       ├── config.py
    │       ├── core.py
    │       ├── __init__.py
    │       └── __pycache__
    ├── tests
    │   ├── test_cli.py
    │   ├── test_config.py
    │   └── test_core.py
```

Test functions should also start with `test_`, for example

```
def test_flux_mag(zeropoint, flux):
```

Let's add pytest to our development dependencies:

```
[project.optional-dependencies]
dev = ["ruff", "mypy", "pytest"]
```

and re-install the project with `pip install -e .[dev]` to install pytest.

You can configure pytest via `pyproject.toml` so there's a single place for all tools:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = [
  "-ra",   # detailed report at the end, including all except passed tests
]
```

With this, a simple:

```
pytest
```

from the project root will run all tests found in `tests/`.

A simple example test

For the `flux_to_mag` function in `candiamazing.utils`:

```
# tests/test_utils.py

import numpy as np
import pytest

from candiamazing.utils import (
   distance_modulus_to_distance,
   distance_to_distance_modulus,
   flux_to_mag,
   mag_to_flux,
)
```

```
@pytest.mark.parametrize("zeropoint", [8.9, 25.0, 30.0])  # run test
with three values for zeropoint
@pytest.mark.parametrize(
    "flux", [1.0, 10.0, 3631.0, np.array([1.0, 10.0, 3631.0])]
)  # run test with multiple values/types for flux
def test_flux_mag(zeropoint, flux):
    # check round trip conversion
    mag = flux_to_mag(flux, zeropoint)
    flux_back = mag_to_flux(mag, zeropoint)

    # These are the actual "tests"
    assert np.all(np.abs(flux - flux_back) < 1e-10), "Round trip flux ->
mag -> flux failed"

    assert np.all(mag == pytest.approx(zeropoint - 2.5 *
np.log10(flux))), (
        "flux_to_mag calculation incorrect"
    )

    assert np.all(flux == pytest.approx(10 ** ((zeropoint - mag) /
2.5))), (
        "mag_to_flux calculation incorrect"
    )
```

A few patterns worth standardising:

- You can use `pytest.approx` for floating-point comparisons, instead of exact equality.
- You can run multiple tests with different values using `@pytest.mark.parameterize(variablename, values)`
- You should check both value and units when using astropy.units.

If your tests require configuration files, or data files, put them in the following structure:

```
tests/
├── test_core.py
├── test_config.py
├── test_cli.py
└── data/
    ├── example_config.toml
    ├── small_input.fits
    └── expected_output.fits
```

You can then access these files during tests by referring to the file path with Pathlib

```
from pathlib import Path
DATA_DIR = Path(__file__).parent / "data"
```

Looking to go deeper and make your life easier? Check out [pytest fixtures]

Test coverage

We use **line coverage** as a rough indicator of how well the code is exercised by tests. It is not perfect, but it catches obvious gaps and discourages untested “dead zones.”

For CANDIAPL projects, the default target is *at least 80% line coverage on the main package, measured by pytest.* For this, we’ll need `pytest-cov` as a dependency:

```
[project.optional-dependencies]
dev = ["ruff", "mypy", "pytest", "pytest-cov"]
```

And we can then configure pyproject.toml to always check coverage while running:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = [
  "-ra",  # detailed report at the end, including all except passed tests
  "--cov=src" # check coverage of code in src/ directory
  "--cov-report=term-missing", # prints uncovered lines at the end
]
```

Then, simply running `pytest` will report the results of testing & coverage.

```
(.venv) osingae:~/OneDrive/postdoc/POSSUM/candiapl-example-software/candiamazing$ pytest
================================================= test session starts =================================================
platform linux -- Python 3.11.5, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/osingae/OneDrive/postdoc/POSSUM/candiapl-example-software/candiamazing
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.0.0
collected 0 items
/home/osingae/OneDrive/postdoc/POSSUM/candiapl-example-software/.venv/lib/python3.11/site-packages/coverage/control.py:957:
 CoverageWarning: No data was collected. (no-data-collected); see https://coverage.readthedocs.io/en/7.13.0/messages.html#w
arning-no-data-collected
  self._warn("No data was collected.", slug="no-data-collected")

================================================== tests coverage ==================================================
_____________________________________ coverage: platform linux, python 3.11.5-final-0 _____________________________________

Name                           Stmts   Miss  Cover   Missing
------------------------------------------------------------
src/candiamazing/__init__.py       2      2     0%   4-6
src/candiamazing/cli.py           19     19     0%   8-52
src/candiamazing/config.py        18     18     0%   3-48
src/candiamazing/core.py           8      8     0%   8-47
------------------------------------------------------------
TOTAL                             47     47     0%
================================================= no tests ran in 0.02s =================================================
```

At this point, we haven’t actually implemented any of the tests yet, so we are at 0% coverage on all files.

Development Workflow and CI

Input required from Mike and Carlo, section in development

- Branching, pull requests, and code review expectations.
- What CI must at least run (format, lint, tests)
- How strict? Coverage?

CI can run Ruff only on the diff vs `main` (e.g. via `git diff --name-only`), so:

- new/modified code must meet docstring rules;
- Untouched files are ignored until someone edits them.

For CI, probably want

- `ruff check` + `ruff format`(no style/lint errors, including docstring rules),
- `mypy` (no type errors, or only known/ignored ones),
- `pytest` (tests pass). At 80% level?

Pull requests should be considered **not ready** if any of these fail.

Enforce the minimum with a .coveragerc file in the repo root?

```
# .coveragerc
[run]
branch = True
source =
    src/candiamazing

[report]
# Enforce minimum total coverage
fail_under = 80
show_missing = True
skip_covered = False
```

Plus this setting in pyproject.toml?

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = [
  "-ra",
  "--strict-markers",
  "--cov=candiamazing",
```

```
  "--cov-report=term-missing",
  "--cov-config=.coveragerc",
]
```

If needed, you can exclude specific files or lines from coverage in the .coveragerc file

```
[run]
omit =
    src/candiamazing/__main__.py
    src/candiamazing/version.py

[report]
exclude_lines =
    pragma: no cover
    if TYPE_CHECKING:
```

But please use `omit` and `pragma: no cover` **sparingly.**

Appendix

Initialise a GitHub repo from the CLI, on Ubuntu

Install GitHub CLI (only once)

```
type -p curl >/dev/null || sudo apt install curl
curl -fsSL https://cli.github.com/packages/githubcli-archive-keyring.gpg |
\
  sudo dd of=/usr/share/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/githubcli-archive-keyring.gpg] \
https://cli.github.com/packages stable main" | \
  sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update
sudo apt install gh
```

Authorise github (only once)

```
gh auth login
```

Create repo from current directory

```
gh repo create USERNAME/reponame --public --source=. --remote=origin --push
```

Then add and commit changes, and do

```
git push --set-upstream origin master
```

Quickstart: setting up a new project

Commands to quickly set up a new project. Replace “candiamazing” with your project name throughout.

1. Create project directory.

```
mkdir candiamazing
cd candiamazing
```

2. Configure the project as a GitHub repo
Section initialise GitHub from CLI

3. Add a standard .gitignore file

```
wget https://raw.githubusercontent.com/github/gitignore/refs/heads/main/Python.gitignore -o .gitignore
```

4. Create a virtual environment

```
python3 -m venv .my_new_venv
```

5. Activate the virtual environment

```
source .my_new_venv/bin/activate
```

6. Set up a typical project structure

```
mkdir -p src/candiamazing/
touch pyproject.toml
touch README.md
touch src/candiamazing/__init__.py
```

7. Set up pyproject.toml as in the example.

8. Install the project as developer in editable mode

```
pip install -e .[dev]
```

9. Check out how to run formatters and linters.

10. Replace the README.md and core code with your own code.

Happy coding!